



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura Licenciatura en Sistemas de Información

Base de Datos II *Grupo N° 11*

Tema 8: Conceptos sobre Procesamiento de Transacciones

INTEGRANTES:

➤ AQUINO, Yamilá	LU: 51386
➤ BAREIRO, Hugo	LU: 36543
➤ COTTI, María de los Angeles	LU: 45390
➤ MAIDANA, Javier	LU: 52726

TABLA DE CONTENIDO

1. INTRODUCCIÓN.....	4
2. DESARROLLO.....	5
2.1 INTRODUCCIÓN AL PROCESAMIENTO DE TRANSACCIONES.....	5
2.1.1 Operaciones de lectura y escritura en una transacción.....	5
2.1.2 Por qué es necesario el control de concurrencia.....	6
2.2 CONCEPTOS DE TRANSACCIONES Y SISTEMAS.....	7
2.2.1 Estados de las transacciones y operaciones adicionales.....	8
2.2.2 La bitácora del sistema.....	9
2.2.3 Punto de confirmación de una transacción.....	10
2.2.4 Puntos de control en la bitácora del sistema.....	11
2.3 PROPIEDADES DESEABLES EN LAS TRANSACCIONES.....	12
2.4 PLANES Y RECUPERABILIDAD.....	14
2.4.1 Planes de transacciones.....	14
2.4.2 Caracterización de planes con base en su recuperabilidad.....	15
2.5 SERIABILIDAD DE LOS PLANES.....	16
2.5.1 Planes en serie, no en serie y seriables por conflictos.....	17
2.5.2 Equivalencia por conflictos y de vistas.....	18
3. CONCLUSIONES.....	20
4. REFERENCIAS BIBLIOGRÁFICAS.....	21

TABLA DE FIGURAS

Ilustración 1 : Diagrama de transición de estados para la ejecución de transacciones....	9
Ilustración 2 : Formas posibles de ordenar las operaciones.....	16

1. INTRODUCCIÓN

El problema de control de concurrencia ocurre cuando múltiples transacciones introducidas por varios usuarios se interfieren de modo que los resultados que se obtienen son incorrectos. En la presente monografía se llevara a cabo un análisis respecto a dichos problemas y además se estudiara sobre la recuperación luego de la falla de una transacción.

Hay que tener en cuenta lo importante que es el control de concurrencia y la recuperación en un sistema de bases de datos, ya que los mecanismos de control de concurrencia y de recuperacion se ocupan principalmente de las ordenes de acceso a la base de datos incluidas en una transacción.

Por tanto, esta monografía estará centrada en los principales conceptos sobre el procesamiento de transacciones, incluyéndose las operaciones de acceso a la base de datos.

2. DESARROLLO [1]

2.1 INTRODUCCIÓN AL PROCESAMIENTO DE TRANSACCIONES

Un criterio para clasificar sistemas de bases de datos es por el número de usuarios que pueden utilizarlos de manera concurrente. Una SGBD es monousuario si un solo usuario a la vez puede utilizarlo, y es multiusuario si muchos usuarios pueden utilizarlo al mismo tiempo.

Los SGBD suelen estar restringidos a algunos sistemas de microcomputador, y casi todos los demás SGBD son multiusuario.

El hecho de que muchos usuarios puedan utilizar los sistemas de computador al mismo tiempo se debe al concepto de multiprogramación, ya que está permite al computador procesar varios programas al mismo tiempo. En el caso de que solo se cuente con una unidad central de procesamiento solo se podrá procesar un programa a la vez.

Los sistemas operativos de multiprogramacion ejecutan algunas ordenes de un programa, luego suspenden ese programa y ejecutan algunas ordenes del siguiente programa, así sucesivamente. Cuando a un programa le llega el turno de usar CPU otra vez, se reanuda su ejecución en el punto en el que se suspendió. Así pues la ejecución concurrente de los programas es en realidad intercalada. Si el computador cuenta con múltiples procesadores en hardware, es posible el procesamiento simultaneo de varios programas, dando lugar a una ocurrencia simultanea no intercalada.

2.1.1 Operaciones de lectura y escritura en una transacción

Las operaciones de acceso a la base de datos que una transacción puede incluir en el nivel de elementos de información y bloques de discos son:

- Leer_elemento (X): Lee un elemento de la base de datos llamado X y lo coloca en una variable de programa. Para simplificar nuestra notación, supondremos que la variable de programa también se llama X.

Su ejecución incluye los siguientes pasos:

- 1) Encontrar la dirección del bloque de disco que contiene el elemento X.
- 2) Copiar ese bloque de disco en un almacenamiento intermedio dentro de la memoria principal.
- 3) Copiar el elemento X del almacenamiento intermedio a la variable de programa llamada X.

➤ **Escribir_elemento (X):** Escribe el valor de la variable de programa X en el elemento de la base de datos llamado X.

Su ejecución incluye los siguientes pasos:

- 1) Encontrar la dirección del bloque de disco que contiene el elemento X.
- 2) Copiar ese bloque de disco en un almacenamiento intermedio dentro de la memoria principal.
- 3) Copiar el elemento X desde la variable de programa llamada X al lugar correcto dentro del almacenamiento intermedio.
- 4) Transferir el bloque actualizado desde el almacenamiento intermedio al disco.

Para tener acceso a la base de datos, una transacción deberá incluir operaciones leer_elemento y escribir_elemento. Los mecanismos de control de concurrencia y de recuperación se ocupan principalmente de las ordenes de acceso a la base de datos incluidas en una transacción.

2.1.2 Por qué es necesario el control de concurrencia

Pueden surgir varios problemas cuando las transacciones concurrentes se ejecutan de manera no controlada. Uno de ellos es el problema de la actualización perdida, la cual ocurre cuando dos transacciones que tiene acceso a los mismos elementos de la base de datos tienen sus operaciones intercaladas de modo que hacen incorrecto el valor de algún elemento.

El problema de actualización temporal (o lectura sucia), esto ocurre cuando una transacción actualiza un elemento de la base de datos y luego la transacción falla por alguna razón. Otra transacción tiene acceso al elemento actualizado antes de que se restaure a su valor original.

El problema del resumen incorrecto, se da si una transacción esta calculando una función agregada de resumen sobre varios registros mientras otras transacciones están actualizando algunos de ellos, puede ser que la función agregada calcule algunos valores antes de que se actualicen y otros después de actualizarse.

Otro problema que puede presentarse es el de la lectura no repetible, en el que una transacción lee un elemento dos veces y otra transacción modifica el elemento entre las dos lecturas. Así la transacción recibe dos valores diferentes en sus dos lecturas del mismo elemento.

Siempre que se introduce una transacción a un SGBD para ejecutarla, el sistema tiene que asegurarse de que todas las operaciones de la transacción se completen con éxito y su efecto quede asentado permanentemente en la base de datos, o que la transacción no tenga efecto alguno sobre la base de datos ni sobre cualquier otra transacción. El SGBD no debe permitir que se apliquen a la base de datos algunas operaciones de una transacción pero no otras operaciones. Esto puede suceder si una transacción falla después de ejecutar algunas de sus operaciones pero antes de ejecutarlas todas.

2.2 CONCEPTOS DE TRANSACCIONES Y SISTEMAS

La ejecución de un programa que incluye operaciones de acceso a la base de datos se denomina transacción de base de datos o simplemente transacción. Si las operaciones de base de datos en una transacción no actualizan datos, sino que solo los leen, se habla de transacción solo de lectura.

2.2.1 Estados de las transacciones y operaciones adicionales

Una transacción es una unidad atómica de trabajo que se realiza por completo o bien no se efectúa en absoluto. Para fines de recuperación, el sistema necesita mantenerse al tanto de cuando la transacción se inicia, termina y se confirma o aborta. A sí, pues el gesto de recuperación se mantiene al tanto de las siguientes operaciones:

- INICIO_DE_TRANSACCION: Esta marca el principio de la ejecución de la transacción.
- LEER o ESCRIBIR: Estas especifican operaciones de lectura o escritura de elementos de la base de datos que se efectúan como parte de una transacción.
- FIN_DE_TRANSACCION: Esta especifica que las operaciones de LEER y ESCRIBIR de la transacción han terminado y marca el límite de la ejecución de la transacción.
- CONFIRMAR_TRANSACCION: Esta indica que la transacción terminó con éxito y que cualquier cambio ejecutada por ella se puede confirmar sin peligro en la base de datos y que no se cancelaran.
- REVERTIR (o ABORTAR): Esta indica que la transacción terminó sin éxito y que cualquier cambio o efecto que pueda haberse aplicado a la base de datos se deben cancelar.

Además de las operaciones antes mencionadas, las técnicas de recuperación también requieren operaciones adicionales:

- DESHACER: Similar a revertir, pero no se aplica a una sola operación y no a una transacción completa.
- REHACER: Esta especifica que ciertas operaciones de transacción deben repetirse para asegurar que todas las operaciones de transacción confirmada se hayan aplicado con éxito a la base de datos.

Una transacción entra en un estado activo inmediatamente después de que inicia su ejecución, y ahí puede emitir operaciones LEER y ESCRIBIR. Cuando la transacción termina, pasa al estado parcialmente confirmado. En este punto, algunas técnicas de control de concurrencia requieren que se efectúen ciertas verificaciones para garantizar que la transacción no interfiera otras transacciones en ejecución.

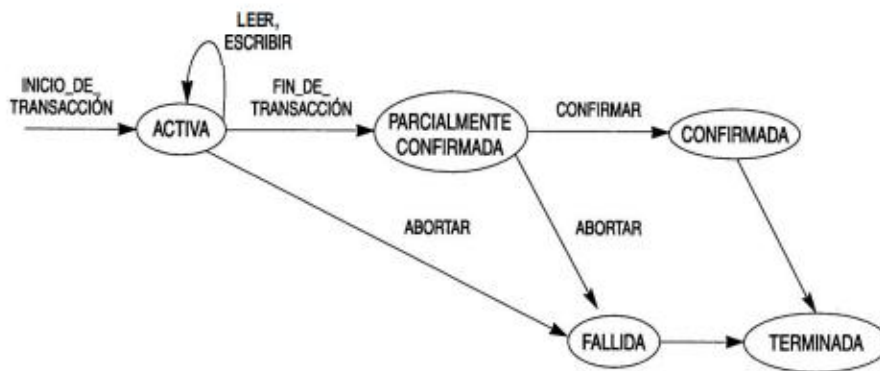


Ilustración 1: Diagrama de transición de estados para la ejecución de transacciones

Por añadidura, algunos protocolos de recuperación deben asegurar que un fallo del sistema no provocara una incapacidad para registrar permanentemente los cambios hechos por la transacción. Una vez realizadas con éxito ambas verificaciones, se dice que una transacción a llegado a un punto de confirmación y pasa al estado confirmado.

Sin embargo, una transacción puede pasar al estado fallido si una de las verificaciones falla o si la transacción se aborta mientras esta en el estado activo. En tal caso es posible que la transacción deba revertirse para anular los efectos de sus operaciones ESCRIBIR sobre la base de datos. El estado terminado corresponde al abandono del sistema por parte de la transacción. Las transacciones fallidas o abortadas pueden reiniciarse posteriormente, ya sea en forma automática o después de reintroducirse como transacciones nuevas.

2.2.2 La bitácora del sistema

Para poderse recuperar de los fallos de transacciones, el sistema mantiene una bitácora que sigue la pista a todas las operaciones de transacciones que afectan los valores de elementos de la base de datos. La bitácora se mantiene en disco, de modo que no la afecta ningún tipo de fallo, mas que los de disco o los catastróficos. Además, la bitácora se respalda periódicamente en cinta para protegerse contra fallos catastróficos.

Hay varios tipos de de entradas que se escriben en la bitácora, considerando a T como un identificador de la transacción único que el sistema genera automáticamente tendremos:

- [inicio_de_transacción, T]: Asienta que se ha iniciado la ejecución de la transacción.
- [escribir_elemento, T, X, valor_anterior, nuevo_valor]: Asienta que la transacción T cambio el valor del elemento de base de datos X de valor-anterior a nuevo-valor.
- [leer_elemento, T, X]: Asienta que la transacción T leyó el valor del elemento de base de datos X.
- [confirmar, T]: Asienta que la transacción T termino con éxito y establece que su efecto se puede confirmar en la base de datos.
- [abortar, T]: Asienta que se aborto la transacción T.

Los protocolos de recuperación que evitan las reversiones en cascada no requieren que las operaciones LEER se asienten en la bitácora del sistema, pero otros protocolos si necesitan estas entradas para la recuperación.

Si el sistema se cae, podemos recuperar la base datos a un estado consistente examinando la bitácora y usando técnicas de recuperación, dado que la bitácora contiene un registro de cada operación ESCRIBIR que altera el valor de algún elemento de la base de datos, es posible deshacer el efecto de estas operaciones ESCRIBIR de una transacción rastreando hacia atrás en la bitácora y restableciendo todos los elementos alterados con una operación ESCRIBIR de una transacción a su valor-anterior.

2.2.3 Punto de confirmación de una transacción

Una transacción llega a su punto de confirmación cuando todas las operaciones que tienen acceso a la base de datos se han ejecutado con éxito y el efecto de todas estas operaciones se ha asentado en la bitácora. Mas allá del punto de confirmación, se dice que la transacción

esta confirmada, y se supone que su efecto se asentó permanentemente en la base de datos. En ese momento la transacción escribe una entrada en la bitácora.

Si ocurre un fallo del sistema, buscamos en la bitácora todas las transacciones que han escrito una entrada en la bitácora pero no han escrito todavía su entrada, es posible que durante el proceso de recuperación estas transacciones tengan que revertirse para cancelar su efecto sobre la base de datos. Las transacciones que ya escribieron su entrada de confirmación en la bitácora también deberán haber asentado en ella todas sus operaciones de ESCRIBIR, pues de lo contrario no estarían confirmadas, su efecto sobre la base de datos puede rehacerse a partir de las entradas de la bitácora.

Con frecuencia se mantiene un bloque del archivo de bitácora en la memoria principal hasta que se llena de entradas y luego se escribe una sola vez en el disco, en vez de escribirlo cada vez que se añade una entrada. Esto ahorra el gasto extra de escribir varias veces en disco el mismo bloque de archivo. Cuando cae el sistema, solo las entradas de bitácora que se han escrito en disco se considera que están en el proceso de recuperación, porque puede haberse perdido el contenido de la memoria principal. Por ello, antes de que una transacción llegue a su punto de confirmación, cualquier porción de la bitácora que no se haya escrito en el disco todavía se deberá grabar. Este proceso se denomina forzar la escritura del archivo de bitácora antes de confirmar una transacción.

2.2.4 Puntos de control en la bitácora del sistema

Otro tipo de entradas en la bitácora es el llamado punto de control. Un registro [punto_de_control] se escribe en la bitácora periódicamente cada vez que el sistema escribe en la base de datos en disco el efecto de todas las operaciones ESCRIBIR de las transacciones confirmadas. De esta forma, todas las transacciones que tengan sus entradas en la bitácora antes de una entrada [punto-de-control] no necesitarán la repetición de sus operaciones ESCRIBIR en caso de una caída del sistema.

El gestor de recuperación de un SGBD debe decidir con que intervalos asentar un punto de control. El intervalo puede medirse en el tiempo o por el numero de transacciones confirmadas desde el ultimo punto de control.

El asentamiento de un punto de control consiste en las siguientes acciones:

- 1) Suspende temporalmente la ejecución de las transacciones.
- 2) Forzar la escritura de todas las operaciones de actualización pertenecientes a transacciones confirmadas del almacenamiento intermedio al disco.
- 3) Escribir un registro [punto-de-control] en la bitácora y forzar la escritura de la bitácora en el disco.
- 4) Reanudar la ejecución de transacciones.

Un registro de punto de control en la bitácora también puede incluir información adicional, como una lista de identificadores de las transacciones activas o las ubicaciones del primer registro y del mas reciente en la bitácora para cada transacción activa.

2.3 PROPIEDADES DESEABLES EN LAS TRANSACCIONES

Las transacciones atómicas deben poseer varias propiedades, las cuales se conocen como ACID. Deben ser impuestas por los métodos de control de concurrencia y de recuperación del SGBD. Las propiedades ACID son:

- 1) Atomicidad: Una transacción es una unidad de procesamiento, o bien se realiza por completo o no se realiza en absoluto.
- 2) Conservación de la consistencia: Una ejecución correcta de la transacción debe llevar a la base de datos de un estado consistente a otro.
- 3) Aislamiento: Una transacción no debe dejar que otras transacciones puedan ver sus actualizaciones antes de que sea confirmada, esta propiedad cuando se hace cumplir estrictamente, resuelve el problema de la actualización temporal y hace innecesarias las reversiones en cascada de las transacciones.

4) Durabilidad o permanencia: Una vez que una transacción ha modificado la base de datos y las modificaciones han confirmado, estas nunca deben perderse por un fallo subsecuente.

La propiedad de atomicidad requiere que ejecutemos una transacción hasta completarla. Es obligación del método de recuperación garantizar la atomicidad.

Se considera que la propiedad de conservación de la consistencia es responsabilidad de los programadores que escriben los programas de bases de datos o del modulo del SGBD que impone las restricciones de integridad.

Un estado de la base de datos es una colección de todos los elementos de información almacenados en la base de datos en un momento dado. Un estado consistente de la base de datos satisface las restricciones especificadas en el esquema, además de cualquier otra restricción que deba cumplirse en la base de datos.

Los programas de bases de datos deben elaborarse a modo de garantizar que, si la base de datos esta en un estado consistente antes de ejecutar la transacción, estará en un estado consistente después de la ejecución completa de la transacción, suponiendo que no hay interferencias con otras transacciones.

El aislamiento lo impone el método de control de concurrencia. Se dice que una transacción tiene aislamiento de grado 0 si no sobrescribe las lecturas sucias de transacciones del nivel mas alto; una transacción con nivel de aislamiento de grado 1 no tiene actualizaciones perdidas, y el aislamiento de grado 2 no tiene actualizaciones perdidas ni lecturas sucias. Por ultimo, el aislamiento de grado 3, al cual también se lo conoce como aislamiento verdadero, tiene las propiedades del grado 2 y lecturas repetibles. La propiedad durabilidad es responsabilidad del método de recuperación.

2.4 PLANES Y RECUPERABILIDAD

Cuando varias transacciones se ejecutan de manera concurrente e intercalada, el orden de ejecución de sus operaciones constituye lo que se conoce como plan o historia de las transacciones.

2.4.1 Planes de transacciones

Un plan o historia P de n transacciones $T_1, T_2, \dots, T_n$, es un ordenamiento para las operaciones de las transacciones sujeto a la restricción de que, para cada transacción T_i que participe en P , las operaciones de T_i en P deben aparecer en el mismo orden en que ocurren en T_i .

Se dice que un plan P de transacciones $T_1, T_2, \dots, T_n$ es un plan completo si se cumplen las siguientes condiciones:

1. Las operaciones de P son exactamente las operaciones de $T_1, T_2, \dots, T_n$, incluidas una operación de confirmar o de abortar como última operación de cada transacción en el plan.
2. Para cualquier par de operaciones de la misma transacción T_i , su orden de aparición en P es el mismo que su orden de aparición en T_i .
3. Para cualquier dos operaciones en conflicto, una de ellas debe ocurrir antes que la otra en el plan.

Las condiciones antes mencionadas permiten que dos operaciones que no estén en conflicto ocurran en el plan sin definir cuál lo haga primero, dando lugar a la definición de un plan como un orden parcial de las operaciones en las n transacciones.

Se debe especificar un orden total en el plan para cualquier par de operaciones en conflicto (condición 3) y para cualquier par de operaciones de la misma transacción (condición 2). La

condición 1 simplemente dice que todas las operaciones en las transacciones deben aparecer en el plan completo. Puesto que toda transacción ha sido confirmada o abortada, un plan completo nunca contendrá transacciones activas.

En general, es difícil encontrar planes completos en un sistema de procesamiento de transacciones, porque continuamente se están introduciendo nuevas transacciones en el sistema.

2.4.2 Caracterización de planes con base en su recuperabilidad

En algunos planes resulta fácil recuperarse de fallos de transacciones, pero en otros dicho proceso puede ser bastante complicado. Por ello, es importante caracterizar los tipos de planes en los cuales es posible la recuperación, así como aquellos en los que la recuperación es relativamente simple. En primer lugar, nos gustaría estar seguros de que, una vez que se ha confirmado una transacción T , nunca será necesario revertirla. Los planes que satisfacen este criterio se denominan planes recuperables.

Se dice que un plan P es recuperable si ninguna transacción T de P se confirma antes de haberse confirmado todas las transacciones T' que han escrito un elemento que T lee. Se dice que una transacción T lee de la transacción T' en un plan P si T' escribe primero algún elemento X y luego T lo lee. En el plan P , T' no deberá haber abortado antes de que T lea el elemento X , y no deberá haber transacciones que escriban X después de que T' lo haya escrito y antes de que T lo lea. En un plan recuperable, ninguna transacción confirmada tiene que revertirse jamás.

Sin embargo, sí es posible que ocurra un fenómeno conocido como reversión en cascada (o aborto en cascada), en el que una transacción no confirmada tiene que revertirse porque leyó un elemento de una transacción fallida. Como la reversión en cascada puede consumir mucho tiempo es importante caracterizar los planes en los que se garantiza que este fenómeno no ocurrirá. Un plan evita la reversión en cascada si toda la transacción del plan solo lee elementos escritos por transacciones confirmadas.

También hay un plan mas restrictivo, llamado plan estricto, en el que las transacciones no pueden leer ni escribir un elemento X en tanto no se haya confirmado (o abortado) la ultima transacción que escribió X. Los planes estrictos simplifican el proceso de recuperar las operaciones de escritura, reduciéndolo a una restauración de la imagen anterior de un elemento de información X, que es el valor que X tenia antes de la operación de escritura abortada.

2.5 SERIABILIDAD DE LOS PLANES

Supongamos que dos usuarios introducen al SGBD las transacciones T_1 y T_2 al mismo tiempo. Si no se permite la intercalación, solo hay dos formas posibles de ordenar las operaciones de las dos transacciones para su ejecución:

1. Ejecutar todas las operaciones de la transacción T_1 seguidas de todas las operaciones de la transacción T_2 .
2. Ejecutar todas las operaciones de la transacción T_2 seguidas de todas las operaciones de la transacción T_1 .

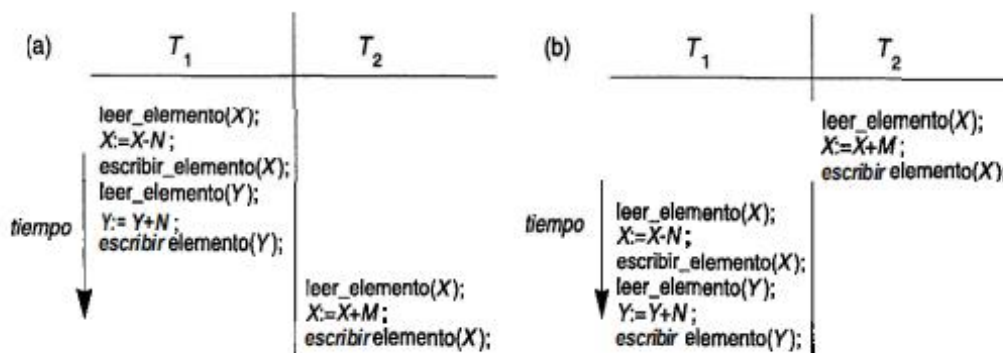


Ilustración 2: Formas posibles de ordenar las operaciones.

Si se permite la intercalación de operaciones, habrá muchas ordenes posibles en que el sistema podrá ejecutar las operaciones individuales de las transacciones.

Un aspecto importante del control de concurrencia es la llamada teoría de la seriabilidad, con la que se intenta determinar cuales planes son correctos y cuales no, e inventar técnicas que solo permitan planes correctos.

2.5.1 Planes en serie, no en serie y serializables por conflictos

Los planes en serie, se denominan así porque, las operaciones de cada transacción se ejecutan de manera consecutiva, sin operaciones intercaladas de la otra transacción. En un plan en serie, se llevan a cabo transacciones completas en serie.

Un plan P es en serie si, por cada transacción T que participa en el plan, todas las operaciones de T se ejecutan consecutivamente en el plan, de lo contrario, el plan no es en serie. Una suposición razonable que podemos hacer, si consideramos que las transacciones son independientes, es que todo plan en serie se considera correcto. La razón es que suponemos que toda transacción es correcta si se ejecuta por si sola, y que transacciones son independientes entre si. Por ello, no importa cual transacción se ejecuta primero. En tanto toda transacción se ejecute de principio a fin sin que la interfieran las operaciones de otras transacciones, obtendremos un resultado final correcto en la base de datos.

El problema con los planes en serie es que limitan la concurrencia o la intercalación de las operaciones. En un plan en serie, si una transacción esta esperando que se complete una operación de E/S, no podremos conmutar el procesador a otra transacción, desperdiciándose así un tiempo valioso de procesamiento de la UCP y haciendo a los planes en serie inaceptables en general.

Un plan P de n transacciones es serializable si es equivalente a algún plan en serie de las mismas n transacciones. Observe que existen $(n)!$ posibles planes en serie de n transacciones y muchos más planes no en serie posibles. Podemos formar dos grupos disjuntos de los planes que no son en serie: los que son equivalentes a uno (o más) de los planes en serie, y por tanto son serializables; y los que no son equivalentes a ningún plan en serie, y por tanto no son serializables.

Decir que un plan no en serie P es serializable equivale a decir que es correcto, porque es equivalente a un plan en serie, que se considera correcto. Dos planes son equivalentes por resultados si producen el mismo estado final en la base de datos.

El enfoque más seguro y general para definir la equivalencia de dos planes consiste en no hacer suposición alguna sobre los tipos de operaciones que intervienen en las transacciones. Para que dos planes sean equivalentes, las operaciones que se aplican a cada uno de los elementos de información afectados por los planes se deben aplicar a ese elemento en el mismo orden en ambos planes.

2.5.2 Equivalencia por conflictos y de vistas

Por lo regular se aceptan dos definiciones de equivalencia de planes: equivalencia por conflictos y equivalencia de vistas. Se dice que dos planes son equivalentes por conflictos si el orden de cualesquier dos operaciones *en conflicto* es el mismo en ambos planes. Si dos operaciones en conflicto se aplican en *diferente orden* en dos planes, el efecto de los planes puede ser diferente, ya sea sobre las transacciones o sobre la base de datos, de modo que los dos planes no son equivalentes por conflictos.

Como vimos antes, decir que un plan P es serializable (por conflictos) (esto es, que P es equivalente (por conflictos) a un plan en serie) es tanto como decir que P es correcto. Sin embargo, ser *serializable* no es lo mismo que ser *en serie*. Un plan en serie representa un proceso ineficiente porque no se permite la intercalación de operaciones de diferentes transacciones. Esto puede dar pie a un bajo aprovechamiento de la UCP mientras una transacción espera una E/S de disco, haciendo al procesamiento bastante más lento. Un plan serializable ofrece los beneficios de la ejecución concurrente sin dejar de ser correcto.

En la práctica, es muy difícil comprobar la serializabilidad de un plan. Casi siempre, el planificador del sistema operativo determina la intercalación de operaciones de transacciones concurrentes.

Si las transacciones se ejecutan a voluntad y luego se comprueba si el plan es serializable, deberemos cancelar el efecto del plan si resulta que no es serializable. Este es un problema grave que hace poco práctico este enfoque. Por ello, la estrategia que se sigue en casi todos los sistemas prácticos es encontrar métodos que garanticen la serializabilidad sin tener que

verificar la seriabilidad de los planes mismos después de haberlos ejecutado. Uno de estos métodos se basa en la teoría de la seriabilidad para determinar protocolos o conjuntos de reglas que, si todas las transacciones individuales los siguen o si un subsistema de control de concurrencia del SGBD los impone, garanticen la seriabilidad de todos los planes en los que participen las transacciones. Así pues, con este enfoque nunca tendremos que ocuparnos de los planes mismos.

La equivalencia de vistas se basa en la idea de que, en tanto cada operación de lectura de una transacción lea el resultado de la misma operación de escritura en ambos planes, las operaciones de escritura de ambas transacciones producirán los mismos resultados. Así pues, se dice que las operaciones de lectura perciben la misma vista en ambos planes.

3. CONCLUSIONES

Luego del análisis llevado a cabo anteriormente acerca de los principales conceptos sobre el procesamiento de transacciones, podemos concluir en lo importante que resultan las transacciones, ya que mediante las operaciones que estas proporcionan se logra el acceso a la base de datos.

Hay tener en cuenta que una transacción se realiza por completo o no se realiza en absoluto, con lo cual se refiere a que está debe llegar a su punto de confirmación el cual se cumple cuando todas las operaciones de acceso se realizan con éxito y los efectos de dichas operaciones quedan asentadas en la base de datos.

Un punto importante a destacar, tiene que ver con la recuperación, ya que el sistema cuenta con una bitácora que contiene todas las operaciones de las transacciones. En caso de que ocurra un fallo en el sistema, se utiliza los datos contenidos en la bitácora, dándose la posibilidad de tener que revertir alguna transacción para cancelar su efecto sobre la base de datos.

4. REFERENCIAS BIBLIOGRÁFICAS

- [1] R. Elmasri y S. Navathe, «CAPÍTULO 17 - Conceptos de procesamiento de transacciones,» de *Sistemas de Bases de Datos Segunda Edición*, Addison-Wesley Iberoamericana, p. 531.